Lab-08: Student Management System

Name: KUSHAL KUMAR C S

SRN: PES1UG23CS320

Section: F

Course: Object Oriented Analysis and Design

1. Problem Statement

The objective of this lab is to develop a web-based Student Management System using the MVC (Model–View–Controller) architecture. The system should allow users to register students by entering their name, email, and course. The application should store this information in a database and display the registered students in a table on the same page. The application should be implemented using Spring Boot for the MVC framework, Hibernate (JPA) for database interaction, and a relational database such as H2 or MySQL.

2. Objective

The aim of this experiment is to understand how MVC architecture works in a real web application. This includes separating the application into Model, View, and Controller layers, implementing basic CRUD operations, and integrating a database using JPA.

3. Technologies Used

- Java (JDK 17)
- Spring Boot
- Spring Data JPA / Hibernate
- Thymeleaf
- H2 Database
- IntelliJ IDEA

4. System Architecture

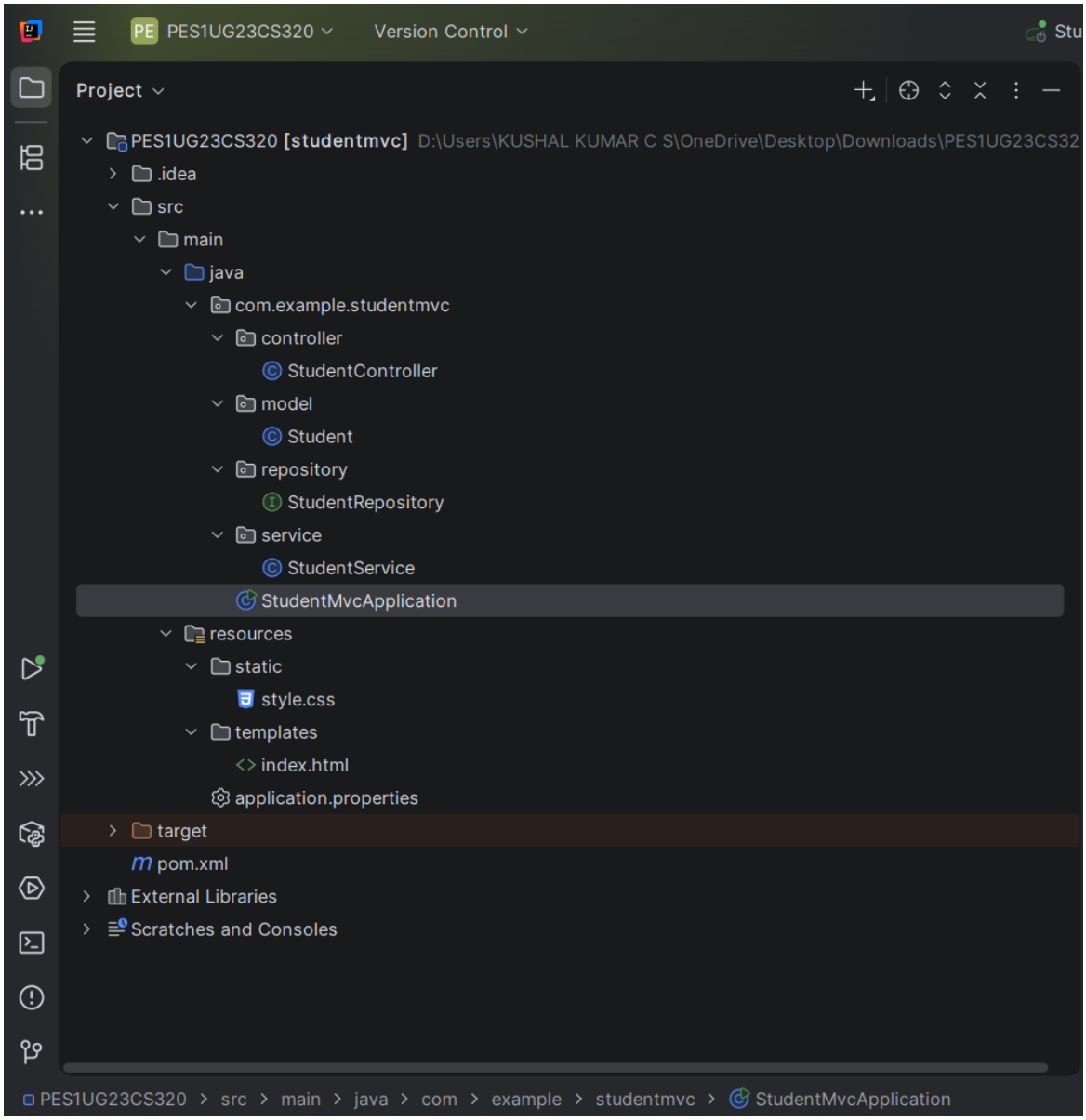
The application follows the MVC architecture where the responsibilities of the program are divided into three main components.

Model: Represents the data and business entities in the application. In this project, the Student class represents the student record with fields such as id, name, email, and course.

View: Responsible for displaying the user interface. Thymeleaf templates are used to render the web page where users can enter student details and view the list of students.

Controller: Handles incoming HTTP requests and communicates with the service layer to process user actions such as adding a new student or retrieving student data.

5. Project Structure



StudentController.java

```java
package com.example.studentmvc.controller;

import ...

@Controller
public class StudentController {

    private final StudentService service;  3 usages

    public StudentController(StudentService service) { this.service = service; }

    @GetMapping("/")
    public String viewHomePage(Model model) {

        model.addAttribute( attributeName: "students", service.getAllStudents());
        model.addAttribute( attributeName: "student", new Student());

        return "index";
    }

    @PostMapping("/api/students")
    public String saveStudent(@ModelAttribute Student student) {

        service.saveStudent(student);

        return "redirect:/";
    }
}
```

Student.java

```java
package com.example.studentmvc.model;

import jakarta.persistence.*;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private String course;

    public Student() {}

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

StudentRepository.java

```java
package com.example.studentmvc.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.studentmvc.model.Student;

import java.util.Optional;

public interface StudentRepository extends JpaRepository<Student, Long> {  3 usages

    Optional<Student> findByEmail(String email);  1 usage

}
```

StudentService.java

```java
package com.example.studentmvc.service;

import com.example.studentmvc.model.Student;
import com.example.studentmvc.repository.StudentRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class StudentService {

    private final StudentRepository repository;  4 usages

    public StudentService(StudentRepository repository) {  no usages
        this.repository = repository;
    }

    public List<Student> getAllStudents() {  1 usage
        return repository.findAll();
    }

    public void saveStudent(Student student) {  1 usage

        if(repository.findByEmail(student.getEmail()).isPresent()) {
            throw new RuntimeException("Email already registered!");
        }

        repository.save(student);
    }
}
```

StudentMvcApplication.java

```java
package com.example.studentmvc;

import ...

@SpringBootApplication
public class StudentMvcApplication {

    public static void main(String[] args) { SpringApplication.run(StudentMvcApplication.class, args); }
}
```

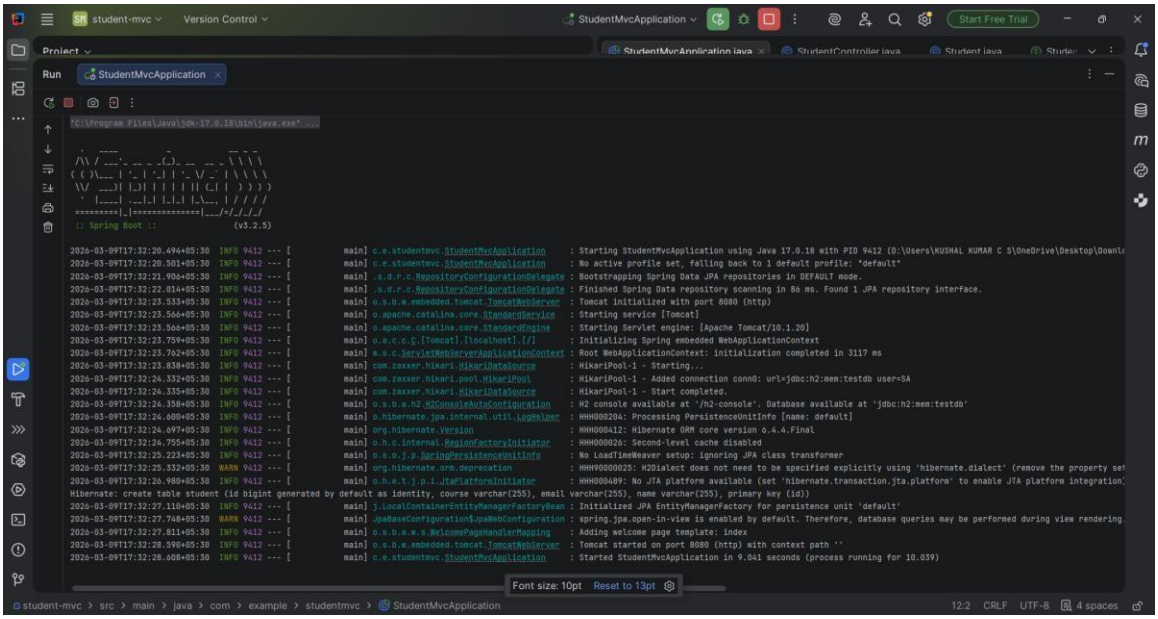
index.html

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <title>Student Management System</title>
    <link rel="stylesheet" th:href="@{/style.css}">
</head>

<body>

<div class="container">

    <h1> Student Management System</h1>

    <div class="card">

        <h2>Register Student</h2>

        <form th:action="@{/api/students}" th:object="${student}" method="post">

            <div class="input-group">
                <label>Name</label>
                <input type="text" th:field="*{name}" placeholder="Enter student name" required>
            </div>

            <div class="input-group">
                <label>Email</label>
                <input type="email" th:field="*{email}" placeholder="Enter email address" required>
            </div>

            <div class="input-group">
                <label>Course</label>
                <input type="text" th:field="*{course}" placeholder="Enter course name" required>
            </div>

            <button type="submit">Register Student</button>

        </form>

    </div>

    <div class="card">
```

index.html

```html
<html xmlns:th="http://www.thymeleaf.org">
<body>
<div class="container">
    </div>

    <div class="card">

        <h2>Student List</h2>

        <table>

            <thead>
            <tr>
                <th>ID</th>
                <th>Name</th>
                <th>Email</th>
                <th>Course</th>
            </tr>
            </thead>

            <tbody>

            <tr th:each="s : ${students}">
                <td th:text="${s.id}"></td>
                <td th:text="${s.name}"></td>
                <td th:text="${s.email}"></td>
                <td th:text="${s.course}"></td>
            </tr>

            </tbody>

        </table>

    </div>

</div>

</body>
</html>
```

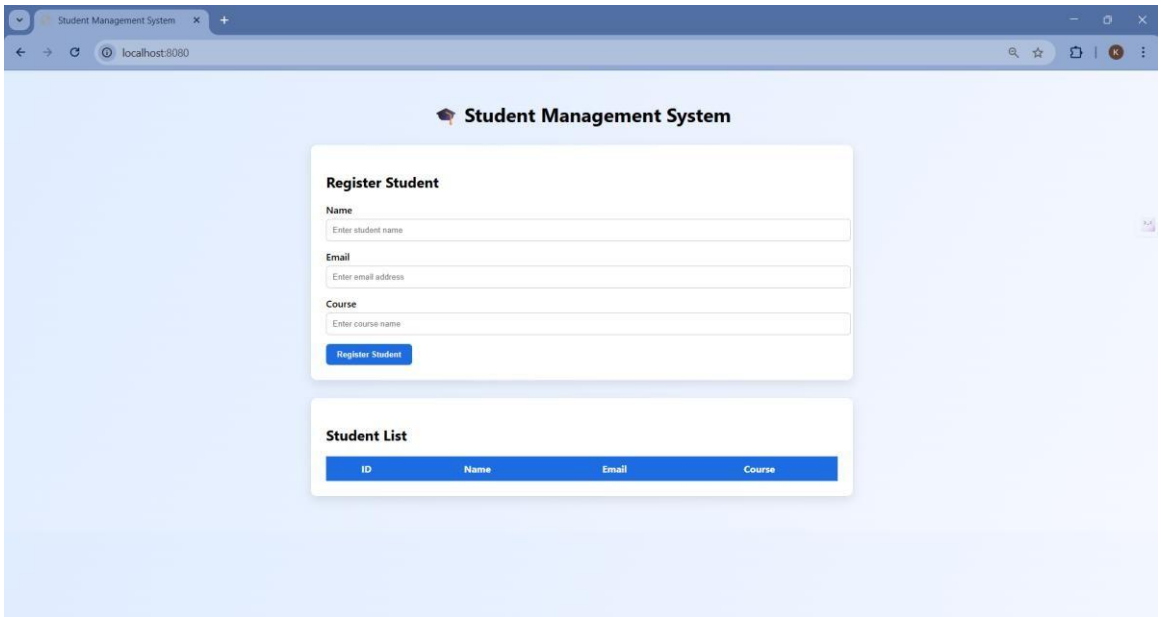
6. Application Execution




```
Hibernate: select s1_0.id,s1_0.course,s1_0.email,s1_0.name from student s1_0
Hibernate: select s1_0.id,s1_0.course,s1_0.email,s1_0.name from student s1_0 where s1_0.email=?
Hibernate: insert into student (course,email,name,id) values (?,?,?,default)
Hibernate: select s1_0.id,s1_0.course,s1_0.email,s1_0.name from student s1_0
Hibernate: select s1_0.id,s1_0.course,s1_0.email,s1_0.name from student s1_0 where s1_0.email=?
Hibernate: insert into student (course,email,name,id) values (?,?,?,default)
Hibernate: select s1_0.id,s1_0.course,s1_0.email,s1_0.name from student s1_0
```

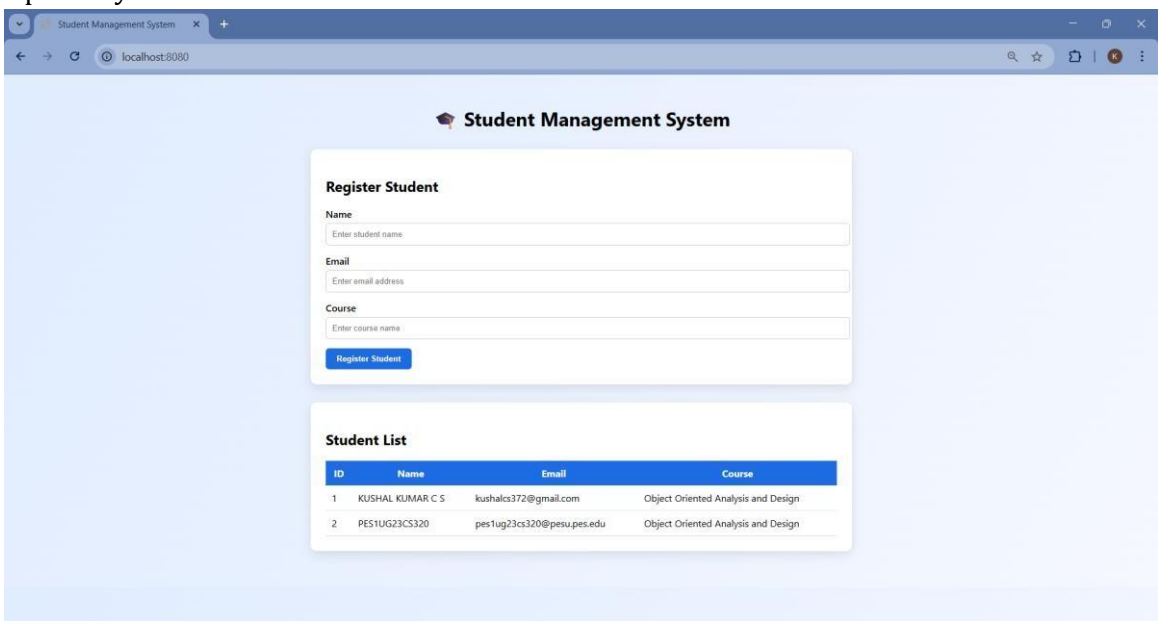
7. User Interface

The application provides a simple web interface where a user can register a student by entering their name, email, and course. Once submitted, the student details are saved in the database and immediately displayed in the student list table.



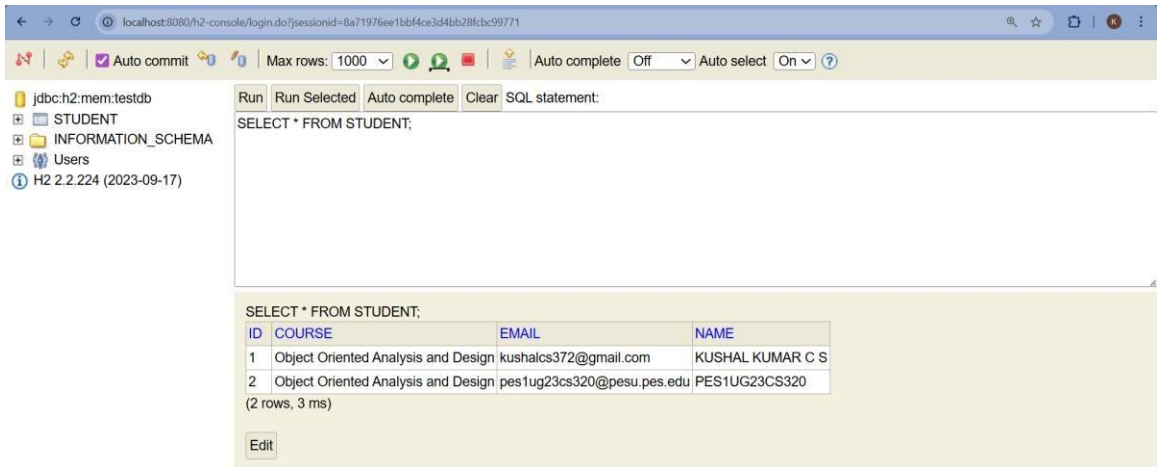
8. Adding Student Record

When the user fills the form and clicks the register button, the controller processes the request and sends the data to the service layer. The service layer interacts with the repository to save the student data into the database.



9. Database Integration

The application uses the H2 in-memory database for storing student information. Hibernate automatically creates the student table based on the entity class. The stored records can be verified using the H2 console.



10. Result

The Student Management System was successfully implemented using the MVC architecture. The application allows users to add student records and view them dynamically on the web page. The data is stored in the database using JPA and Hibernate, demonstrating the separation of concerns between the Model, View, and Controller layers.

This experiment helped in understanding the practical implementation of MVC architecture using Spring Boot and how different layers interact with each other to build a maintainable web application.